Project Report on De-herding Recommender Systems

Team 9

Shiven Bajpai[1], Shivram S[2], Yadlapally Shreedhanvi[3], Charitha Sri[4]

[1]AI24BTECH11030 [2]AI24BTECH11031 [3]AI24BTECH11036 [4]AI24BTECH11014

Abstract

This report investigates herding in recommender systems. It focuses on herding due to popularity bias and conformity bias, and analyzes methods designed to counteract them. We review two classes of approaches: re-ranking methods that adjust exposure, and causal methods that attempt to separate preference from bias during training. We also discuss the role of base recommenders and the metrics used to evaluate trade-offs between accuracy, diversity, and calibration.

The second part presents the implementation. We describe the dataset, preprocessing choices, and training procedures for each method. We then compare models across standard ranking and de-herding metrics, highlighting our observations and tradeoffs. We also discuss possible future improvements to our implementation.

Team Contributions

- **Shiven Bajpai** researched conformity bias and the MACR causal inference framework, studying the role of counterfactual inference in isolating true user preference from popularity-driven signals, and helped in debugging implementations.
- **Shivram S** conducted the broader literature review beyond the assigned papers, and was responsible for the full implementation pipeline, including Matrix Factorization, LightGCN, MACR, and CP re-ranking as well as model training and the comparative evaluation across all methods.
- **Yadlapally Shreedhanvi** researched and documented the xQuAD and Calibrated Popularity re-ranking frameworks, surveyed evaluation metrics for popularity bias (ARP, ACLT, UPD), and implemented the xQuAD re-ranking algorithm.

- **Charitha Sri** extended the CP framework with Wasserstein distance in place of Jensen-Shannon divergence, implemented Nash Social Welfare for fairness evaluation, developed the modified Bayesian Personalized Ranking base model, and implemented the Smooth xQuAD variant along with additional evaluation metrics.

CONTENTS

I. Introduction and Problem Statement

Recommender systems tend to exhibit a phenomenon called **herding**, where user preference and model predictions converge toward a narrow subset of items. Among the many biases responsible for this, we will focus on popularity bias and conformity bias. **Popularity bias** is a phenomenon where popular items (the ”Short Head”) are recommended disproportionately often, while niche items (the ”Long Tail”) are ignored [1]. This creates a significant ”herding effect” [1].

A. The Impact of Popularity Bias

The consequences of this bias are three-fold:

- **For Users:** Individuals with niche interests fail to discover relevant content because the system defaults to popular items.
- **For Businesses:** The economic value of digital catalogs often resides in the long tail; pushing only popular items wastes the diversity of the catalog.

- **For Fairness:** New producers and indie artists face a ”rich get richer” scenario where they cannot break through the algorithmic barrier.

B. The Herding Feedback Loop and History Contamination

Popularity bias is exacerbated by herding, where the system pushes popular items, training users to interact with them, which in turn provides biased data back to the algorithm. This leads to **history contamination**. If a user is only ever shown blockbusters, their interaction history will suggest they have zero preference for the long tail, even if they would have enjoyed niche content had it been presented.

C. Conformity Bias

A related but distinct concept is **conformity bias** [2]. This occurs when users behave similarly to a group even if it contradicts their personal judgment. For example, a user might watch a trending movie not because they like the genre, but because it is popular [3]. Standard training fails here because the model learns to reproduce these interactions, assigning high scores to items based on popularity rather than actual fit.

II. Literature Review: Re-ranking Approaches

Earlier fixes for popularity bias involved post-processing steps to diversify recommendation lists. These steps depend on the scores assigned by a base recommender system, and rerank items to mitigate herding.

A. xQuAD: Explicit Query Aspect Diversification

xQuAD was originally designed for web-search[4] and was used to address situations where a user's query might have multiple meanings (e.g. Java, the programming language, v.s. Java, the island). xQuad was adapted for recommender systems to balance accuracy and exposure of niche items.

At each step of the iterative process, the algorithm selects the item v from the set of remaining items $R \setminus S$ that maximizes the following score:

$$\text{score} = (1-\lambda)P(v|u) + \lambda \sum_{c \in \{\Gamma, \Gamma'\}} P(c|u)P(v|c) \prod_{i \in S} (1 - P(i|c, S))$$

The score consists of the following terms:

- **Accuracy Term** ($P(v|u)$): Represents the likelihood of user u being interested in item v, provided by the base recommender.
- **Trade-off Parameter** (λ): A hyperparameter controlling the strength of popularity bias mitigation.
- **Categories** (Γ, Γ'): The item catalog is divided into the long-tail (Γ) and the short-head (Γ').
- **Personalization** ($P(c|u)$): The user's historical interest in items from category c (e.g., preference for niche vs. popular content).
- **Category Coverage** ($P(v|c)$): An indicator function that is 1 if $v \in c$ and 0 otherwise.
- **Redundancy Penalty** ($\prod_{i \in S}(1-P(i|c, S))$): This term reduces the score for items belonging to a category c that is already represented in S.

The diversity term $P(v, S'|u)$ is computed as the marginal likelihood over the item categories:

$$P(v, S'|u) = \sum_{d \in \{\Gamma, \Gamma'\}} P(v, S'|d)P(d|u)$$

Assuming that items are independent of each other given the categories, the probability $P(v, S'|d)$ is expanded as:

$$P(v, S'|d) = P(v|d)P(S'|d) = P(v|d) \prod_{i \in S} (1 - P(i|d, S))$$

Theare are two methods of calculating the probability $P(i|d, S)$:

1) **Binary xQuAD**: $P(i|d, S)$ is an indicator function. The product term is 1 if category d has not been covered in S yet, and 0 otherwise.
2) **Smooth xQuAD**: $P(i|d, S)$ is the ratio of items in S that belong to category d, allowing for a more gradual adjustment of scores.

B. Calibrated Popularity (CP) and UPD

To address the limitations of xQuAD, **Calibrated Popularity (CP)**[5] focuses on **User Popularity Deviation (UPD)**[5]. UPD measures how well the recommendation list matches a user's historical taste distribution. Items are partitioned into three groups based on their popularity in the training data:

- **Head items (H):** The items making up the top 20% of total interactions.
- **Tail items (T):** The items making up the bottom 20% of interactions.
- **Mid items (M):** Items in between that receive the remaining 60% of interactions.

To measure the bias, we define two discrete probability distributions over the categories $C = \{H, M, T\}$ for each user u. The distribution P reflects the popularity of items in the user's historical profile ρ_u, and Q reflects the popularity of items in the recommendation list L_u.

The profile propensity $p(c|u)$ is defined as:

$$p(c|u) = \frac{\sum_{i \in \rho_u} r(u,i)\mathbf{1}(i \in c)}{\sum_{c' \in C} \sum_{i \in \rho_u} r(u,i)\mathbf{1}(i \in c')}$$

The recommendation propensity $q(c|u)$ is defined as:

$$q(c|u) = \frac{\sum_{i \in L_u} \mathbf{1}(i \in c)}{\sum_{c' \in C} \sum_{i \in L_u} \mathbf{1}(i \in c')}$$

where $r(u,i)$ is the rating for item i.

The **User Popularity Deviation (UPD)** for a user group g is the average Jensen-Shannon divergence $\mathfrak{J}$ between the historical distribution P and the recommendation distribution Q:

$$UPD(g) = \frac{1}{|g|} \sum_{u \in g} \mathfrak{J}(P, Q)$$

The Jensen-Shannon divergence is a symmetric measure of statistical distance between two distributions, where lower values indicate that the recommendations better match the user's historical interest in terms of item popularity.

The **Calibrated Popularity (CP)** algorithm is a re-ranking method that takes an initial list of size m from a base model and selects a final list L_u of size n ($m \gg n$). The goal is to maximize a weighted sum of predicted relevance and popularity calibration:

$$L_u^* = \arg \max_{L_u, |L_u|=n} (1-\lambda) \cdot Rel(L_u) - \lambda \cdot \mathfrak{J}(P, Q(L_u))$$

where:

- $Rel(L_u)$ is the sum of predicted scores for items in L_u.
- $\mathfrak{J}(P, Q(L_u))$ is the Jensen-Shannon divergence between the user's profile and the recommendation list.
- $\lambda \in [0, 1]$ is the trade-off parameter. When $\lambda = 0$, the algorithm maximizes accuracy; when $\lambda = 1$, it focuses entirely on matching the user's popularity preference.

III. Literature Review: Causal Inference Approaches

A. The MACR Framework

Model-Agnostic Counterfactual Reasoning (MACR) attempts to solve the bias at the root by modeling the causal paths of user interaction [3].

MACR assumes that a recommendation score Y is influenced by three distinct causal paths: user-Item matching ($U, I \rightarrow Y$), item popularity bias ($I \rightarrow Y$), and user conformity bias ($U \rightarrow Y$).

The prediction score is fused using a sigmoid-based multiplicative mechanism:

$$\hat{y}_{u,i} = \sigma(y_{ui}) \cdot \sigma(y_i) \cdot \sigma(y_u)$$

where:

- y_{ui} is the personalized matching score from a base recommender.
- y_i is the item popularity branch (typically a learnable bias or MLP on item i).
- y_u is the user activity/conformity branch (typically a learnable bias or MLP on user u).

To ensure each branch captures its intended effect, MACR employs a multi-task learning objective. The model is trained to minimize the sum of the main recommendation loss and two auxiliary losses for the popularity branches:

$$\mathcal{L} = \mathcal{L}_{main} + \alpha \mathcal{L}_{item} + \beta \mathcal{L}_{user}$$

The individual losses are defined as:

$$\mathcal{L}_{main} = \text{BCE}(\hat{y}_{u,i}, y)$$

$$\mathcal{L}_{item} = \text{BCE}(\sigma(y_i), y), \quad \mathcal{L}_{user} = \text{BCE}(\sigma(y_u), y)$$

where $y \in \{0, 1\}$ represents the historical interaction.

B. Counterfactual Inference

The goal of debiasing is to calculate the **Total Indirect Effect (TIE)**, which removes the direct effect of item popularity.

$$\text{TIE} = \text{TE} - \text{NDE}$$

MACR assumes that the direct effect is proportional to the user conformity and item popularity coefficients with constant c, giving us

$$\text{Score}_{u,i} = \sigma(y_{ui})\sigma(y_i)\sigma(y_u) - c\sigma(y_i)\sigma(y_u)$$

By choosing an appropriate constant c, the model ranks items based on their personalized relevance while discounting the influence of global popularity.

IV. Literature Review: Base Recommender Models

All approaches discussed above require a base recommender model to provide the recommendation score. There are two primary candidates for a recommender: **Matrix Factorization (MF)** and **LightGCN**. Both models learn latent representations of users and items from implicit feedback, but differ fundamentally in how they model interactions and propagate collaborative signals.

A. Matrix Factorization (MF)

Matrix Factorization models user-item interactions as inner products in a shared latent space. Given a user set U and item set I, MF learns embeddings $\mathbf{p}_u \in \mathbb{R}^d$ for each user $u \in U$ and $\mathbf{q}_i \in \mathbb{R}^d$ for each item $i \in I$, such that the predicted preference is:

$$\hat{y}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i. \tag{1}$$

For binary ratings, MF is typically optimized using pairwise ranking losses such as Bayesian Personalized Ranking (BPR):

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)} \log \sigma(\hat{y}_{ui} - \hat{y}_{uj}) \tag{2}$$

where (u, i, j) denotes that user u interacted with item i but not item j, and $\sigma(\cdot)$ is the sigmoid function.

MF is computationally efficient, scales well to large datasets, and provides strong baseline performance. However, user preferences are inferred only from direct interactions.

B. LightGCN

LightGCN extends MF by modeling the user-item interaction graph. It treats the data as a bipartite graph where edges represent observed interactions. Let $\mathbf{e}_u^{(0)}$ and $\mathbf{e}_i^{(0)}$ denote initial embeddings (analogous to MF). LightGCN performs K layers of propagation:

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}(u)} \frac{1}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(i)|}} \mathbf{e}_i^{(k)}, \quad \mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}(i)} \frac{1}{\sqrt{|\mathcal{N}(i)||\mathcal{N}(u)|}} \mathbf{e}_u^{(k)}, \tag{3}$$

where $\mathcal{N}(\cdot)$ denotes the set of neighbors in the interaction graph.

The final embedding is computed as a weighted sum of embeddings across layers:

$$\mathbf{e}_u = \sum_{k=0}^{K} \alpha_k \mathbf{e}_u^{(k)}, \quad \mathbf{e}_i = \sum_{k=0}^{K} \alpha_k \mathbf{e}_i^{(k)}. \tag{4}$$

Predictions are again computed via inner product:

$$\hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i. \tag{5}$$

By aggregating over multiple hops, LightGCN captures *higher-order collaborative signals*, allowing information to propagate through indirect connections.

V. Evaluation Metrics for De-herding

A. Standard Accuracy Metrics

- **Recall@K**: Measures the proportion of relevant items in the test set R_u that are successfully included in the top-K recommendation list L_u.

$$\text{Recall@K} = \frac{1}{|U_t|} \sum_{u \in U_t} \frac{|L_u \cap R_u|}{|R_u|}$$

- **Precision@K**: Represents the fraction of recommended items in the top-K list L_u that are relevant to the user.

$$\text{Precision@K} = \frac{1}{|U_t|} \sum_{u \in U_t} \frac{|L_u \cap R_u|}{K}$$

- **NDCG@K (Normalized Discounted Cumulative Gain)**: Evaluates ranking quality by penalizing relevant items appearing lower in the list. It is the ratio of Discounted Cumulative Gain (DCG) to the Ideal DCG (IDCG).

$$\text{DCG@K} = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

- **Hit Rate (HR@K)**: A binary metric checking if at least one relevant item is present in the top-K recommendations.

$$\text{HR@K} = \frac{1}{|U_t|} \sum_{u \in U_t} \mathbf{1}(L_u \cap R_u \neq \emptyset)$$

B. De-herding Performance Metrics

Measuring the success of de-herding requires specific metrics that quantify popularity bias and catalog coverage.

- **ARP (Average Recommendation Popularity)**: Measures the average popularity of recommended items based on their frequency $\phi(i)$ in the training set.

$$\text{ARP} = \frac{1}{|U_t|} \sum_{u \in U_t} \frac{\sum_{i \in L_u} \phi(i)}{|L_u|}$$

- **APLT (Average Percentage of Long Tail)**: Measures the average percentage of items in recommendation lists that belong to the long-tail set Γ.

$$\text{APLT} = \frac{1}{|U_t|} \sum_{u \in U_t} \frac{|L_u \cap \Gamma|}{|L_u|}$$

- **ACLT (Average Coverage of Long Tail)**: Evaluates the total exposure or breadth of the long-tail catalog across all users.

$$\text{ACLT} = \frac{1}{|U_t|} \sum_{u \in U_t} \sum_{i \in L_u} \mathbf{1}(i \in \Gamma)$$

- **UPD (User Popularity Deviation)**: Evaluates the discrepancy between the average popularity of items in a user's profile H_u and the items in their recommendation list L_u.

$$UPD = \mathfrak{J}(H_u, L_u)$$

- **Exposure Disparity**: It is the ratio of popular item (short head) recommendations to niche item (long tail) recommendations.

$$\text{Exposure Disparity} = \frac{\text{No. of short head items}}{\text{No. of long tail items}}$$

VI. Wasserstein Distance

We also tried replacing the Jenson-Shannon divergence used in CP reranking with **Wasserstein Distance**. The Wasserstein distance between two probability distributions P and Q is given by

$$W(P,Q) = \int_{-\infty}^{\infty} |F_P(x) - F_Q(x)| dx$$

Our motivation for using this over Jenson-Shannon divergence was that

1) It is robust to zero-probability bins.
2) It does not require absolute continuity, and works even when supports are disjoint.
3) It is more computationally efficient to calculate than Jenson-Shannon divergence for 1D distributions.

VII. Implementation Details

A. Dataset and Pre-processing

For all experiments, we utilized the **MovieLens-100k** dataset. To adapt the 5-star rating system for de-herding and MACR training, we binarized the interactions. A rating of ≥ 4 was classified as a positive interaction, while lower ratings were treated as negative feedback.

We noted that the test dataset was inherently biased towards popular items. To ensure that the models do not exploit this bias, we **balanced the test set** by sampling it uniformly acorss items, preventing popular items from having a higher number of interactions.

B. Base Model Selection and Training

We evaluated two primary candidates for our base recommender: Matrix Factorization (MF) and LightGCN. While LightGCN incorporates graph convolutions to capture higher-order relationships, it required approximately 10x more training time and showed lower NDCG and Recall results on our specific dataset compared to MF. Therefore, we used **Matrix Factorization** as the foundation for all subsequent debiasing experiments.

There were two candidate loss functions for training the base Matrix Factorization model. The first one was to model recommendation as a binary classification task, and use **binary cross-entropy** to score the model's predictions.

The second approach was to use **Bayesian Personalized Ranking (BPR)** as the training objective. We tried both approaches, and BPR consistently produced models with higher recall than cross-entropy.

Both models were trained with the Adam optimizer ($\eta = 10^{-3}$) for 50 epochs.

C. MACR and Multi-Task Training

The MACR framework was implemented as a wrapper over the base MF model. We considered three modes of training the model

1) **Jointly training MACR and MF**: We first tried training both MF and MACR together using the BCE loss function over 50 epochs. However this gave us sub-optimal NDCG.
2) **Training MF, then MACR**: We then tried training MF first for 50 epochs, and then jointly training them for 20 epochs. However, the NDCG dropped during the joint training phase, giving us results similar to purely joint training.
3) **Two-stage Fine Tuning**: We trained MF using BPR loss for 50 epochs, then froze the weights and trained MACR on top for 20 epochs using BCE loss. This gave us the best results.

We also tried using BPR loss for MACR, but this produced worse results than BCE.

D. Re-ranking Strategies

We implemented two major re-ranking algorithms on top of the base model:

- **Smooth xQuAD:** This variant was chosen over Binary xQuAD because it allows the diversity bonus to decrease gradually rather than dropping to zero immediately, which is more effective for dense datasets like MovieLens.
- **Calibrated Popularity (CP):** We used **Wasserstein Distance** instead of Jensen-Shannon divergence for calibration. The Wasserstein variant was implemented to be more robust to the zero-probability bins often found in sparse user histories.

We also tested various reranking strategies on metrics such as Exposure disparity and Nash Social Welfare.

VIII. Experimental Results

A. Comparison of De-Herding Methods

Our results highlight the effectiveness of different de-herding tools across accuracy and diversity metrics (see Table below).

Model	NDCG@10	ARP ↓	APLT ↑	UPD ↓
Base MF	0.0347	135.25	0.4451	0.1600
MF + xQuAD	0.0324	120.64	0.5184	0.0889
MF + CP	0.0329	122.36	0.4952	0.0744
Base MF + MACR	0.0197	84.31	0.6870	0.2383
MF + MACR + xQuAD	0.0208	89.18	0.5505	0.0980
MF + MACR + CP	0.0207	85.10	0.5710	**0.0032**

TABLE I: Comparison of De-herding Methods (Base: Matrix Factorization)

We observe that:

- **Effect of MACR:** We observe that MACR is a very aggressive de-herding tool, significantly reducing Average Recommendation Popularity (ARP) from 135.25 to 84.31 and increasing the Average Percentage of Long Tail items (APLT) to 0.6870.
- **Trade-off between accuracy and diversity:** We observe a notable drop in NDCG@10 when using MACR (from 0.0347 to $\sim$0.02), indicating that removing conformity bias removes interaction signals that models typically rely on for accuracy.
- **Deviation from User Preferences:** The combination of **MACR + CP Reranking** achieved a near-perfect User Popularity Deviation (UPD) of 0.0032. This suggests the system is almost perfectly matching the user's natural tolerance for popular vs. niche content.

B. Comparison of Reranking Systems

We also compared reranking methods on top of a base model (MF, trained with BPR loss over 30 epochs). Various metrics for the reranked recommender systems are listed in Table 2.

We observed that **Smooth xQuAD** achieved the highest long-tail coverage and lowest exposure disparity. It reduces disparity from 3142 to 495, which is a $6.3\times$ improvement. However, this

Method	nDCG@10	Pre@10	Rec@10	F1@10	LT Cov.	Ser@10
Baseline	0.2117	0.1549	0.1627	0.1295	0.0075	0.9989
LT-Reg	0.2167	0.1552	0.1672	0.1308	0.0201	0.9989
Smooth xQuAD	0.2117	0.1549	0.1627	0.1295	**0.0426**	0.9989
CP-Wasserstein	0.2142	0.1561	0.1644	0.1309	0.0050	0.9989
Ensemble	0.2117	0.1549	0.1627	0.1295	0.0075	0.9989

TABLE II: Performance comparison across methods.

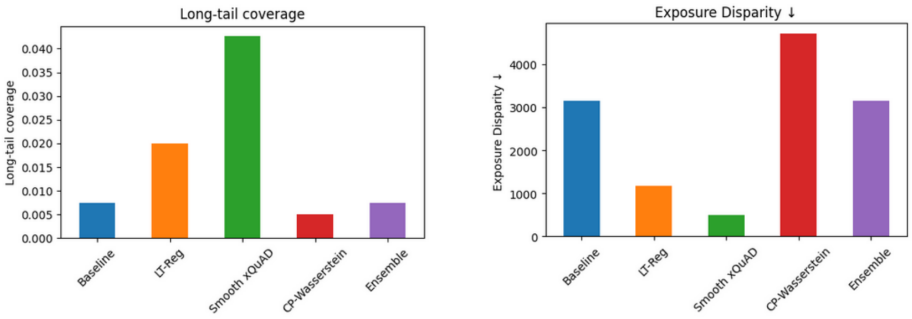

Fig. 1: Left: Long tail coverage. Right: Exposure disparity

can still be improved, as popular items are still recommended $495\times$ more often than long-tail items.

Calibrated Popularity with **Wasserstein Distance** lowers average poularity, but fails to promote long-tail items. The long-tail coverage drops to 0.5%, which is lower than the baseline.

C. Fairness and Social Welfare

We used **Nash Social Welfare (NSW)** to measure if long-tail exposure was distributed fairly across users. Nash Social Welfare computes the utility for a population by taking the geometric mean of a metric over the population.

$$NSW = \left(\prod_{u=1}^{n} v_u \right)^{1/n}$$

Nash Social Welfare penalizes systems that ignore some users. Having a user with a very low value of a metric beings down the NSW of the population. Hence it tends to improve the worst-off user. The Nash Social Welfare for Long-Tail Coverage for various users is illustrated in Fig. 2.

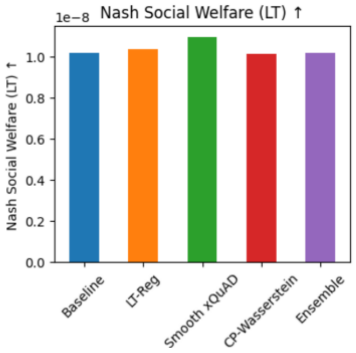

Fig. 2: Nash Social Welfare of Long-Tail Coverage (higher is better)

Smooth xQuAD achieved the highest NSW, confirming it is the most effective method for ensuring that long-tail discovery is shared equitably among the entire user base rather than just a few niche enthusiasts.

IX. Conclusion and Future Scope

Our results show that **re-ranking methods** such as Smooth xQuAD improve long-tail exposure with minimal impact on accuracy, while **causal methods** (MACR) more aggressively reduce bias but at a clear cost to ranking performance. **Calibrated Popularity** (CP) aligns recommendations with user history, but might reinforce existing bias if the history is contaminated.

No single method dominates. improving diversity, accuracy, and preference alignment cannot be achieved simultaneously. Effective de-herding depends on where intervention is applied—ranking, training, or data—and what trade-offs are acceptable.

There are a lot of experiments that could be conducted in the future to improve upon our work:

- Rigorous tuning of hyperparameters to improve performance (α, β, c for MACR; λ for xQuAD and CP)
- Having an adaptive λ for each user based on how much they value relevance over diversity.
- Test different divergence metrics for CP and also explore other models for debiasing.
- Test our model on a real-world platform, where the model will have to adapt as new users and items are added.

REFERENCES

[1] Himan Abdollahpouri et al. Managing popularity bias in recommender systems with personalized re-ranking. In *Proceedings of the 32nd FLAIRS Conference*, 2019.

[2] Jiawei Chen et al. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems (TOIS)*, 41(3):1–46, 2023.

[3] Tianxin Wei et al. Model-agnostic counterfactual reasoning for eliminating popularity bias in recommender system. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1841–1850, 2021.

[4] Rodrygo LT Santos et al. Exploiting query aspects for diversifying search results. In *Proceedings of the 33rd international ACM SIGIR conference*, pages 467–474, 2010.

[5] Himan Abdollahpouri et al. User-centered evaluation of popularity bias in recommender systems. In *Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization*, 2021.